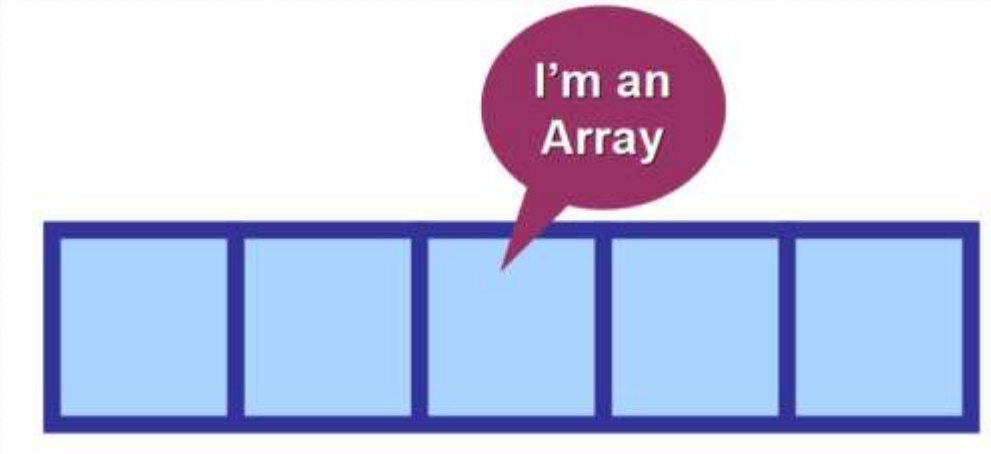
Data Structure Using C & C++

Array



Array

An array is a collection of one or more values of the same type having continuous memory allocation. Each value is called an element of the array. The elements of the array share the same variable name but each element has its own unique index number (also known as a subscript).

Arrays can be

one-dimensional

multidimensional.

An array of **one dimension** is known as a **one-dimensional array, 1-D array** or **Linear array**

An array of **two dimensions** is known as a **two-dimensional array** or **2-D array.**

An array of **n dimensions** is known as a **n-dimensional array** or **n-D array**

Linear Array

- A linear array is a list of finite number n of homogeneous data elements such that:
  - The elements of the array are **referenced respectively by an index set** consisting of n consecutive numbers
  - The elements of the array are stored respectively in **successive memory locations.**

Linear Array

- The number n of elements is called the **length or size** of the array.
- Length or the number of data elements of the array can be obtained from the index set by

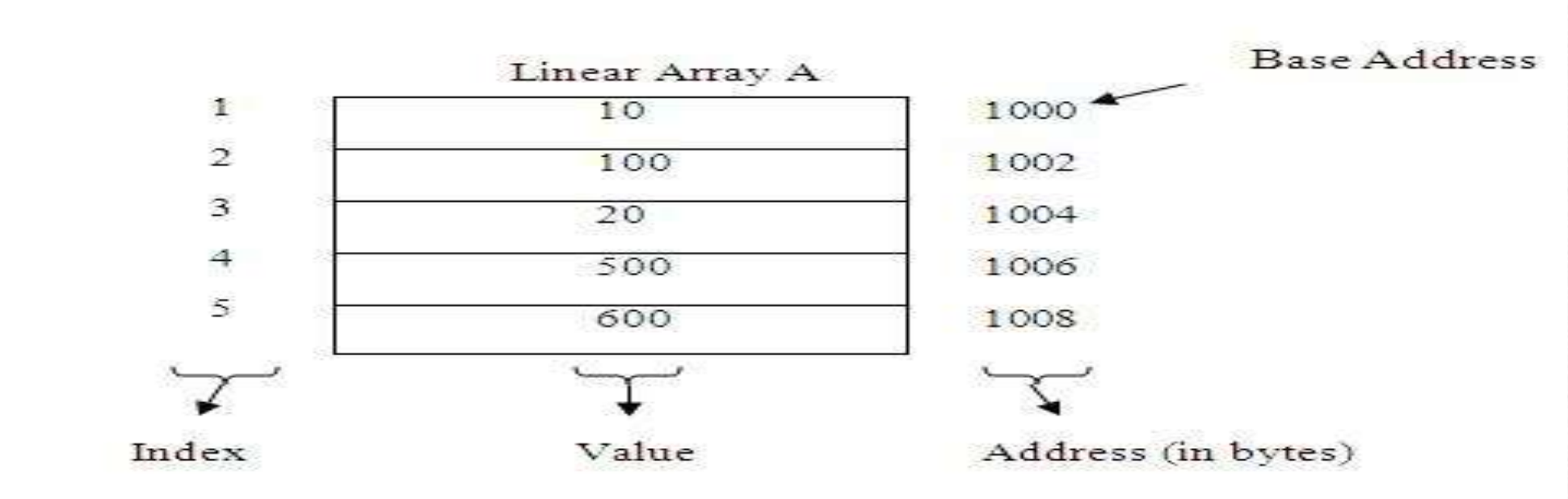
$$\textbf{Length} = \textbf{UB} - \textbf{LB} + 1$$

where UB is the largest index called the **upper bound** and

LB is the smallest index called the **lower bound** of the arrays.

Linear Array

- int A[5] = {10, 100, 20, 500, 600};



- Length = UB- LB+1
  = 5 -1+1
  = 5

Linear Array

- Element of an array A may be denoted by –
  - **Subscript notation** A1, A2, , …. ,
  - **Parenthesis notation** A(1), A(2), …. , A(n)
  - **Bracket notation** A[1], A[2], ….. , A[n]

The number K in A[K] is called **subscript** or an **index** and A[K] is called a **subscripted variable**

Address Calculation in Linear Array:

- Let A be a linear array in the memory of the computer
- LOC(A[K]) = address of the element A[K] of the array A
- The element of A are stored in the successive memory cells
- **Computer does not need to keep track of the address of every element of A**
- Only the address of the first element of the array is stored denoted by Base(A) called the **base address** of A

Address Calculation in Linear Array:

- **LOC(A[K]) = Base(A) + w(K – LB)**

where w is the number of words per memory cell of the array A [w is the size of the data type]

Address Calculation in Linear Array:

Example:

- Given the base address of an array **B[1300…..1900]** as 1020 and size of each element is 2 bytes in the memory. Find the address of **B[1700]**.
- **Solution:**
- The given values are: B = 1020, LB = 1300, W = 2, K = 1700
- **Address of A [K] = B + W * (K – LB)**
- = 1020 + 2 * (1700 – 1300)
  = 1020 + 2 * 400
  = 1020 + 800
  = 1820 **[Ans]**

Operation on Linear Structure

- **Traversal** : Processing each element in the list
- **Search** : Finding the location of the element with a given value or the record with a given key
- **Insertion**: Adding a new element to the list
- **Deletion**: Removing an element from the list
- **Sorting** : Arranging the elements in some type of order
- **Merging** : Combining two list into a single list

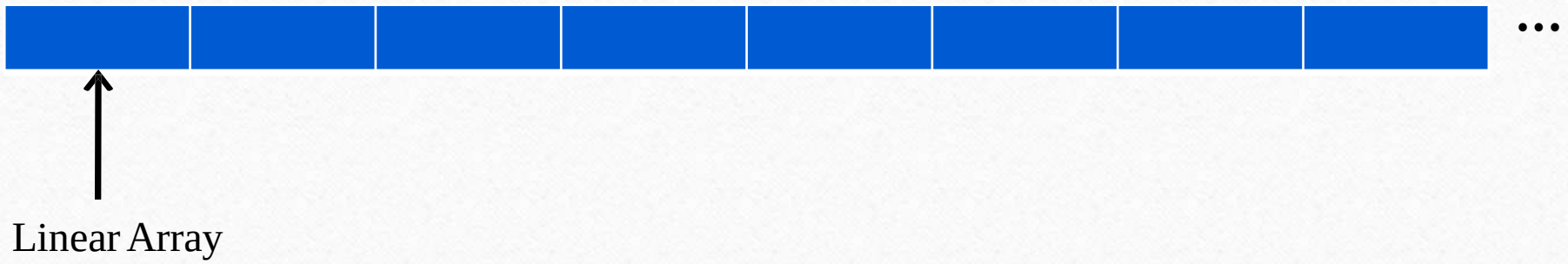
Traversing Linear Arrays

- Traversing is accessing and processing each element of the data structure exactly once.

Linear Array

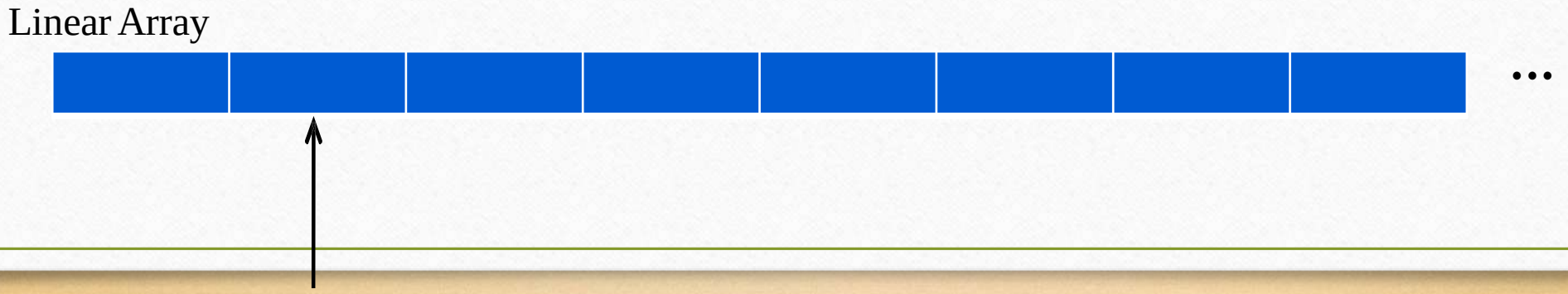
Traversing Linear Arrays

- Traversing is accessing and processing each element of the data structure exactly once.



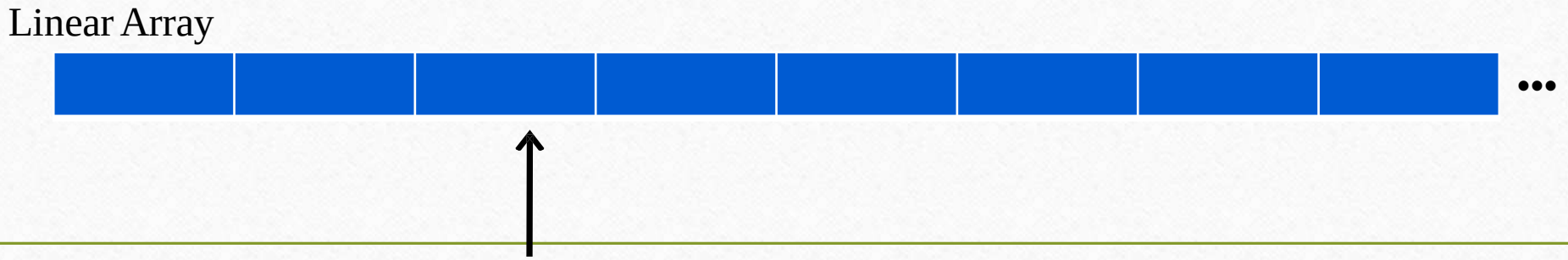
Traversing Linear Arrays

- Traversing is accessing and processing each element of the data structure exactly once.



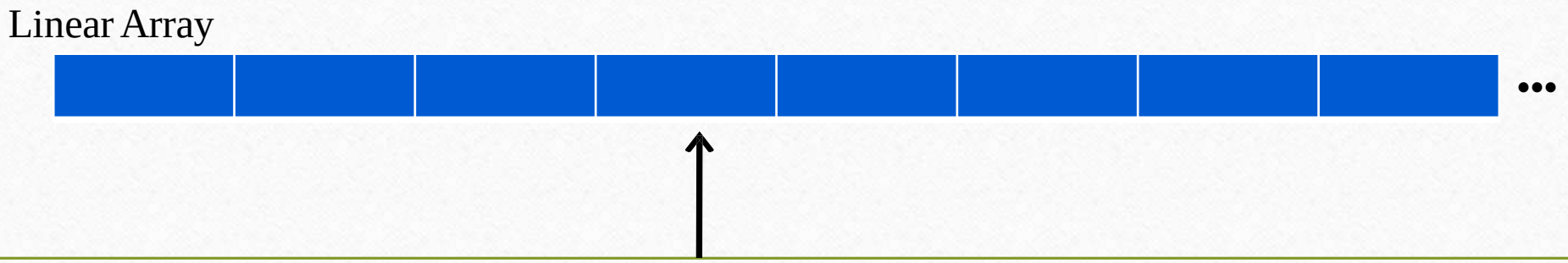
Traversing Linear Arrays

- Traversing is accessing and processing each element of the data structure exactly once.



Traversing Linear Arrays

- Traversing is accessing and processing each element of the data structure exactly once.



Traversing Linear Arrays

- Traversing is accessing and processing each element of the data structure exactly once.

Linear Array

1. Repeat for K = LB to UB Apply PROCESS to LA[K] [End of Loop]
2. Exit

```
for( i=0;i<size;i++)
  printf("%d",LA[i]);
```

Inserting and Deleting

- **Insertion**: Adding an element
  - Beginning
  - Middle
  - End
- **Deletion**: Removing an element
  - Beginning
  - Middle
  - End

Insertion

1	**Brown**
2	Davis
3	Johnson
4	Smith
5	Wagner
6	
7	
8	

1	**Brown**
2	Davis
3	Johnson
4	Smith
5	Wagner
6	Ford
7	
8	

Insert Ford at the End of Array

Insertion

1	**Brown**
2	Davis
3	Johnson
4	Smith
5	Wagner
6	
7	
8	

1	**Brown**
2	Davis
3	Johnson
4	Smith
5	
6	Wagner
7	
8	

1	**Brown**
2	Davis
3	Johnson
4	
5	Smith
6	Wagner
7	
8	

1	**Brown**
2	Davis
3	Ford
4	Johnson
5	Smith
6	Wagner
7	
8	

Insert Ford as the 3rd Element of Array

Insertion is not Possible without loss of data if the array is FULL

Insertion Algorithm

- **INSERT (LA, N , K , ITEM)** [LA is a linear array with N elements and K is a positive integers such that K ≤ N. This algorithm inserts an element ITEM into the Kth position in LA]

1. [Initialize Counter] Set J := N
2. Repeat Steps 3 and 4 while J ≥ K
3. [Move the Jth element downward] Set LA[J + 1]:= LA[J]
4. [Decrease Counter] Set J := J -1
5. [Insert Element] Set LA[K] := ITEM
6. [Reset N] Set N := N +1;
7. Exit

Deletion

1	**Brown**
2	Davis
3	Ford
4	Johnson
5	Smith
6	Taylor
7	Wagner
8	

1	**Brown**
2	Davis
3	Ford
4	Johnson
5	Smith
6	Taylor
7	
8	

Deletion of Wagner at the End of Array

Deletion

1	**Brown**
2	Ford
3	Johnson
4	Smith
5	Taylor
6	Wagner
7	
8	

No data item can be deleted from an empty array

Deletion

1	**Brown**
2	Davis
3	Ford
4	Johnson
5	Smith
6	Taylor
7	Wagner
8	

1	**Brown**
2	
3	Ford
4	Johnson
5	Smith
6	Taylor
7	Wagner
8	

1	**Brown**
2	Ford
3	
4	Johnson
5	Smith
6	Taylor
7	Wagner
8	

1	**Brown**
2	Ford
3	Johnson
4	
5	Smith
6	Taylor
7	Wagner
8	

Deletion of Davis from the Array

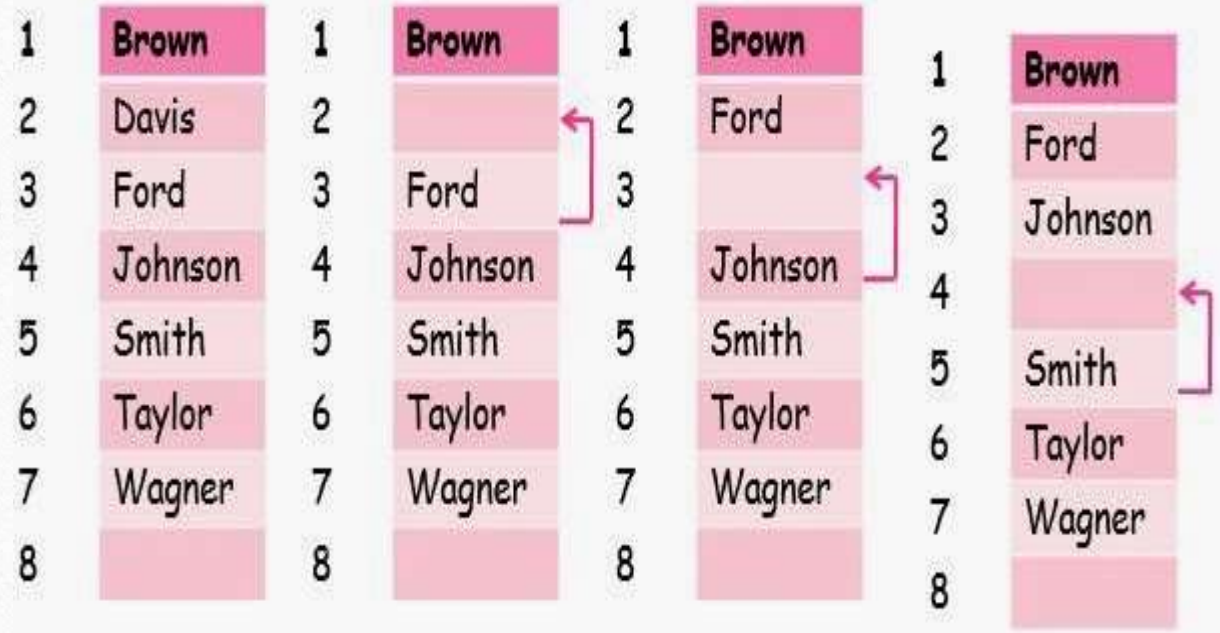
Deletion Algorithm

- **DELETE (LA, N , K , ITEM)** [LA is a linear array with N elements and K is a positive integers such that $K \leq N$. This algorithm deletes K^{th} element from LA]

1. Set ITEM := LA[K]
2. Repeat for J = K+1 to N:

   [Move the Jth element upward] Set LA[J-1]:= LA[J]
3. [Reset the number N of elements] Set N := N - 1;
4. Exit

Deletion from middle / beginning of the array



- Consider **A** is a linear array with **N** elements and **K** is a positive integer such that **K<=N**. Following is the algorithm to delete an element available at the Kth position of A.

STEP I : START
STEP II: Set J = **K**
STEP III : Repeat steps 4 and 5 while J<N
STEP IV: **Set A[J] = A[J +1]**
STEP V: Set J = **J+1**
STEP VI: Set N= **N-1**
STEP VI: STOP

Deletion from middle / beginning of the array

```
#include <stdio.h>
void main()
{
  int A[] = {1,3,5,7,8};
   int i , j , k = 3, n = 5;
  printf("The original array elements are");
  for(i = 0; i<n; i++)
      printf("A[%d] = %d \n", i, A[i]);
j = k;
while( j < n)
 {
    A[j-1] = A[j];
     j = j + 1;   }
 n = n -1;
printf("The array elements after deletion :\n");

for(i = 0; i<n; i++)
      printf("A[%d] = %d \n", i, A[i]);
  }
```

Output:

```
The original array elements are :
 A[0] = 1
 A[1] = 3
A[2] = 5
 A[3] = 7
 A[4] = 8
The array elements after deletion :
 A[0] = 1
A[1] = 3
 A[2] = 7
 A[3] = 8
```

ThankYou

Data Structure Using C & C++

IMS GHAZIABAD
UNIVERSITY COURSES CAMPUS
Estd. 1990

Sparse Matrices

$$A_{sparse} = \begin{bmatrix} 0 & A_{12} & A_{13} & 0 & 0 \\ 0 & A_{22} & 0 & 0 & 0 \\ 0 & 0 & A_{33} & 0 & 0 \\ 0 & 0 & A_{43} & A_{44} & 0 \\ 0 & A_{52} & 0 & 0 & 0 \end{bmatrix}$$

Sparse Matrices

- If a matrix contains more number of ZERO values than NON-ZERO values

Such matrix is known as sparse matrix.

$$\begin{bmatrix} 1.1 & 0 & 0 & 0 & 0 & 0 & 0.5 \\ 0 & 1.9 & 0 & 0 & 0 & 0 & 0.5 \\ 0 & 0 & 2.6 & 0 & 0 & 0 & 0.5 \\ 0 & 0 & 7.8 & 0.6 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1.5 & 2.7 & 0 & 0 \\ 1.6 & 0 & 0 & 0 & 0.4 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0.9 & 1.7 \end{bmatrix}$$

- **Sparse matrix is a matrix which contains very few non-zero elements.**

Sparse Matrices

- **A lot of space gets wasted when a sparse matrix is represented with a 2-dimensional array.**
- **For example, consider an integer matrix of size 100 X 100 containing only 10 non-zero elements.**

In this matrix, only 10 spaces are filled with non-zero values and remaining spaces of the matrix are filled with zero.

That means, totally we allocate 100 X 100 X 2 = 20000 bytes of space to store this integer matrix. And to access these 10 non-zero elements we have to make scanning for 10000 times..

Sparse Matrices Representation

- So, instead of storing zeroes with non-zero elements, we only store non-zero elements.

Sparse Matrix Representations can be done in many ways following are two common representations:

1. **Triplet Representation (Array representation)**
2. **Linked list representation**

Triplet Representation (Array Representation)

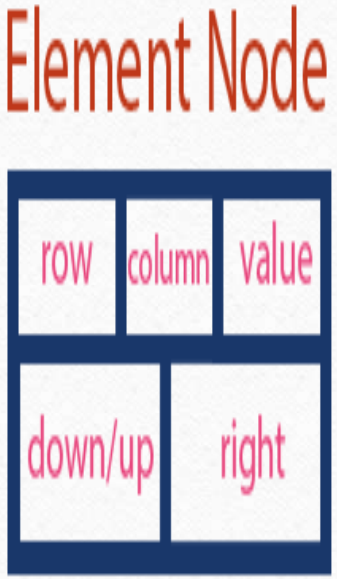
- In this representation, we consider only non-zero values along with their row and column index values.
- The 0th row stores the total number of rows, total number of columns and the total number of non-zero values in the sparse matrix.
- Eg. a matrix of size 5 X 6 containing 6 number of non-zero values. can be represented as shown :

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 9 & 0 \\ 0 & 8 & 0 & 0 & 0 & 0 \\ 4 & 0 & 0 & 2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 5 \\ 0 & 0 & 2 & 0 & 0 & 0 \end{bmatrix}$$

Rows	Columns	Values
5	6	6
0	4	9
1	1	8
2	0	4
2	3	2
3	5	5
4	2	2

Linked Representation

- In linked representation, we use a linked list data structure to represent a sparse matrix.
- In this linked list, we use two different nodes namely
- **header node:** consists of three fields
- **element node**. consists of five fields.



Linked Representation

0 0 0 0 9 0

0 8 0 0 0 0

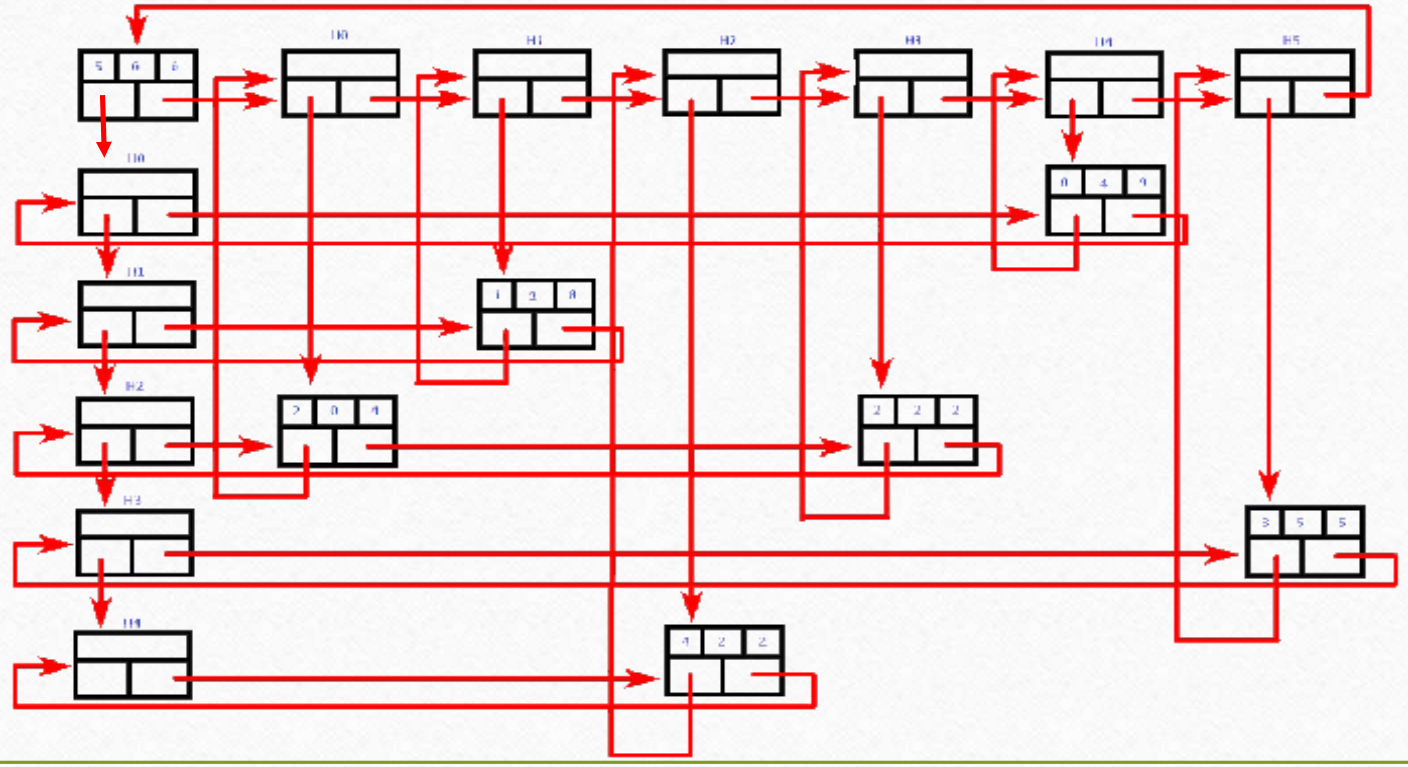
4 0 0 2 0 0

0 0 0 0 0 5

0 0 2 0 0 0

- Nodes H0, H1,..., H5 indicates the header nodes which are used to represent indexes.
- Remaining nodes are used to represent non-zero elements in the matrix, except the very first node which is used to represent abstract information of the sparse matrix (i.e., It is a matrix of 5 X 6 with 6 non-zero elements).

  In this representation, in each row and column, the last node right field points to its respective header node.



Sparse Matrices

- **Lower triangular :** A square matrix is called lower triangular if all the entries **above the main diagonal are zero.**
- **Upper triangular:** A sparse matrix is called upper triangular if all the entries **below the main diagonal are zero**.
- A **triangular matrix** is one that is either **lower triangular** or **upper triangular**

$$\begin{pmatrix} 2 & 0 & 0 & 0 \\ 3 & 5 & 0 & 0 \\ 8 & 6 & 7 & 0 \\ 4 & 1 & 4 & 9 \end{pmatrix} \qquad \begin{pmatrix} 2 & 1 & 8 & 6 \\ 0 & 5 & 3 & 4 \\ 0 & 0 & 7 & 2 \\ 0 & 0 & 0 & 9 \end{pmatrix}$$

Lower Upper

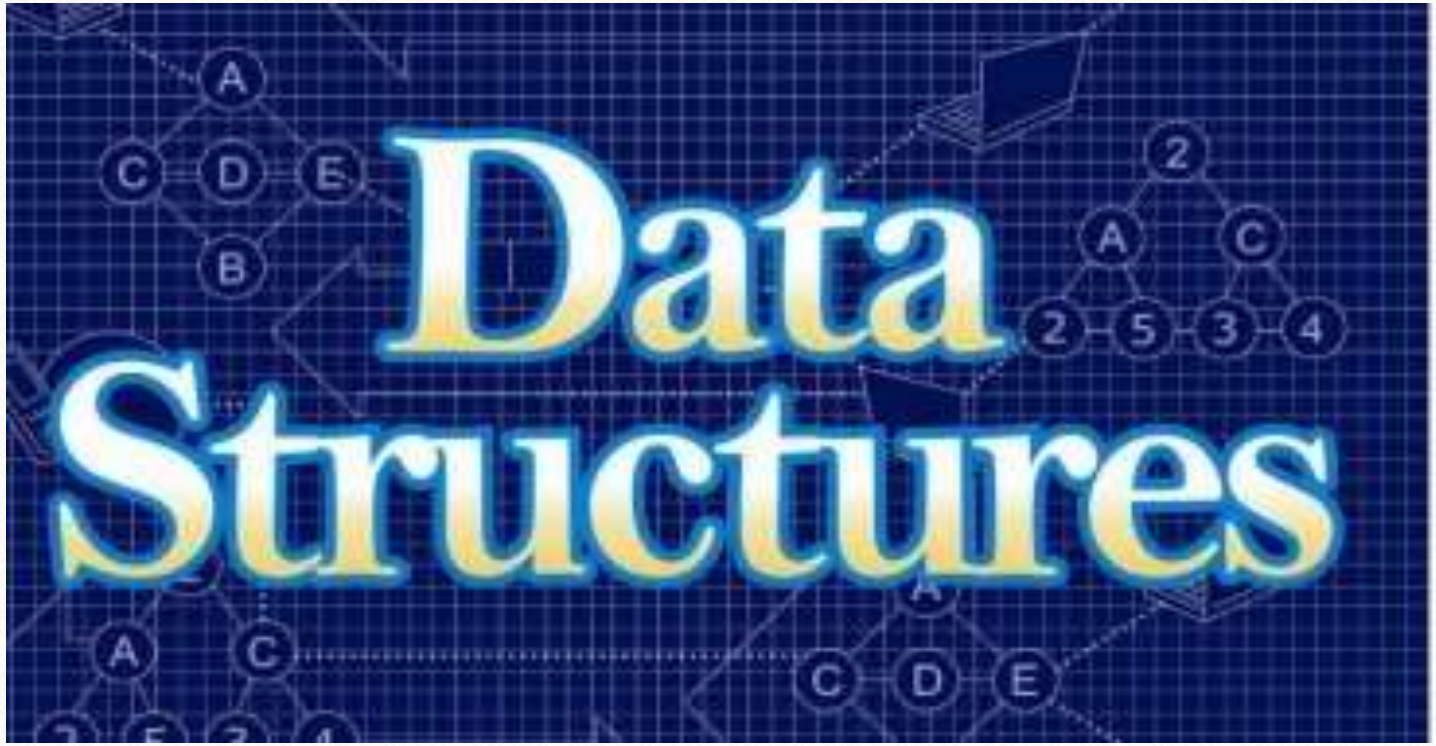
Sparse Matrices

- **Tridiagonal matrices** are the matrices which are having non-zero elements on the diagonal, super diagonal and sub diagonal. All the rest of the elements are zeros.

$$\Lambda = \begin{bmatrix} 1 & -2 & 0 & 0 & 0 \\ 1 & -1 & 3 & 0 & 0 \\ 0 & 5 & 0 & -1 & 0 \\ 0 & 0 & -2 & 1 & 4 \\ 0 & 0 & 0 & 1 & 3 \end{bmatrix}$$

ThankYou

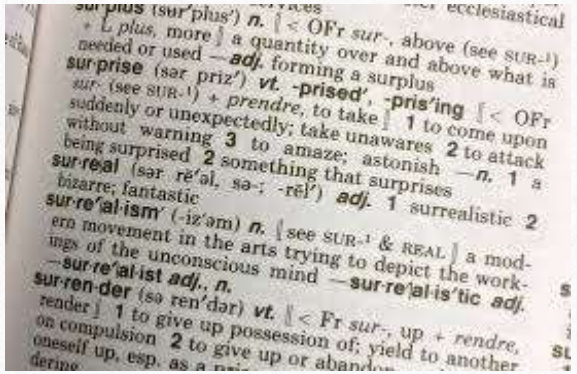
Data Structure Using C & C++
Introduction



Definition

- **Data:** Collection of **raw facts**.
- **Data structure** is representation of the logical relationship existing between individual elements of data.
- **Data structure** is a specialized format for organizing and storing data in memory that considers not only the elements stored but also their relationship to each other.

- **Data** may be organized in many different ways.
- **"Data Structure** is the Logical or mathematical model of a particular organization of data."
- **Definition -**
  - A data structure is a way of organizing all data items that considers not only the elements stored but also their relationship to each other so that they can be used efficiently.



sur·plus (sur′plus′) n. [< OFr sur-, above (see SUR-¹) + L plus, more] a quantity over and above what is needed or used —adj. forming a surplus

sur·prise (sər priz′) vt. -prised′, -pris′ing [< OFr sur- (see SUR-¹) + prendre, to take] 1 to come upon suddenly or unexpectedly; take unawares 2 to attack without warning 3 to amaze; astonish —n. 1 a being surprised 2 something that surprises

sur·real (sər rē′əl, sə-; -rel′) adj. 1 surrealistic 2 bizarre; fantastic

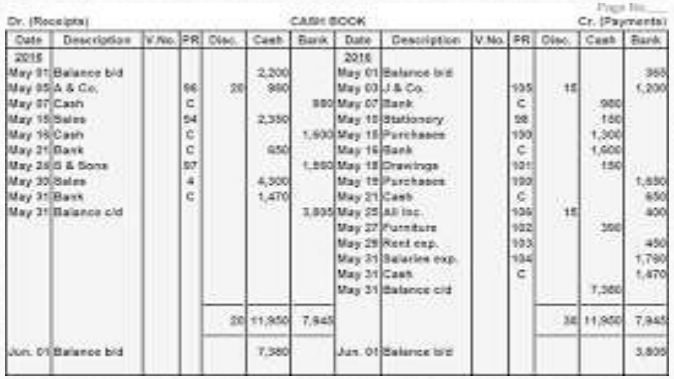
sur·re′al·ism′ (-iz′əm) n. [see SUR-¹ & REAL] a modern movement in the arts trying to depict the workings of the unconscious mind —sur·re′al·ist adj., n.

sur·ren·der (sə ren′dər) vt. [< Fr sur-, up + rendre, render] 1 to give up possession of; yield to another on compulsion 2 to give up or abandon oneself up, esp. as a pris-

ecclesiastical

Dictionary

MAP



Dr. (Receipts) CASH BOOK Page No.____ Cr. (Payments)

Date	Description	V.No.	PR	Disc.	Cash	Bank	Date	Description	V.No.	PR	Disc.	Cash	Bank
2016							2016						
May 01	Balance b/d				2,200		May 01	Balance b/d					360
May 03	A & Co.		66	20	980		May 03	J & Co.		135	15		1,200
May 07	Cash		C			980	May 07	Bank		C		980	
May 15	Sales		54		2,350		May 15	Stationery		58		150	
May 16	Cash		C			1,900	May 16	Purchases		130		1,300	
May 21	Bank		C		650		May 16	Bank		C		1,600	
May 28	G & Sons		57			1,550	May 18	Drawings		101		150	
May 30	Sales		4		4,300		May 19	Purchases		130			1,650
May 31	Bank		C		1,470		May 21	Cash		C			650
May 31	Balance c/d					3,895	May 25	Ali Inc.		136	15		400
							May 27	Furniture		102		360	
							May 29	Rent exp.		133			450
							May 31	Salaries exp.		134			1,790
							May 31	Cash		C			1,470
							May 31	Balance c/d				7,380	
				35	11,950	7,945					35	11,950	7,945
Jun. 01	Balance b/d				7,380		Jun. 01	Balance b/d					3,895

Cash Book

- **Introduction**
- Data structure affects the design of both structural & functional aspects of a program.

Program=algorithm + Data Structure

- You know that a algorithm is a step by step procedure to solve a particular function.
- That means, algorithm is a set of instruction written to carry out certain tasks & the data structure is the way of organizing the data with their logical relationship retained.
- To develop a program of an algorithm, we should select an appropriate data structure for that algorithm.
- Therefore algorithm and its associated data structures form a program.

We talk about Data Structures as:

1. Mathematical /Logical Models or Abstract Data Type
2. Implementation

Abstract Data Type

- An **abstract data type**, sometimes abbreviated **ADT**, is a logical description of how we view the data and the operations that are allowed without regard to how they will be implemented. This means that we are concerned only with what the data is representing and not with how it will eventually be constructed.

Classification of Data Structure

- Data structure are normally divided into two broad categories:
  - Primitive Data Structure
  - Non-Primitive Data Structure
- **Primitive**
  - Used to represent a single value.
  - Also known as simple data types
  - integer, real ,character and Boolean.

DATA STRUCTURE

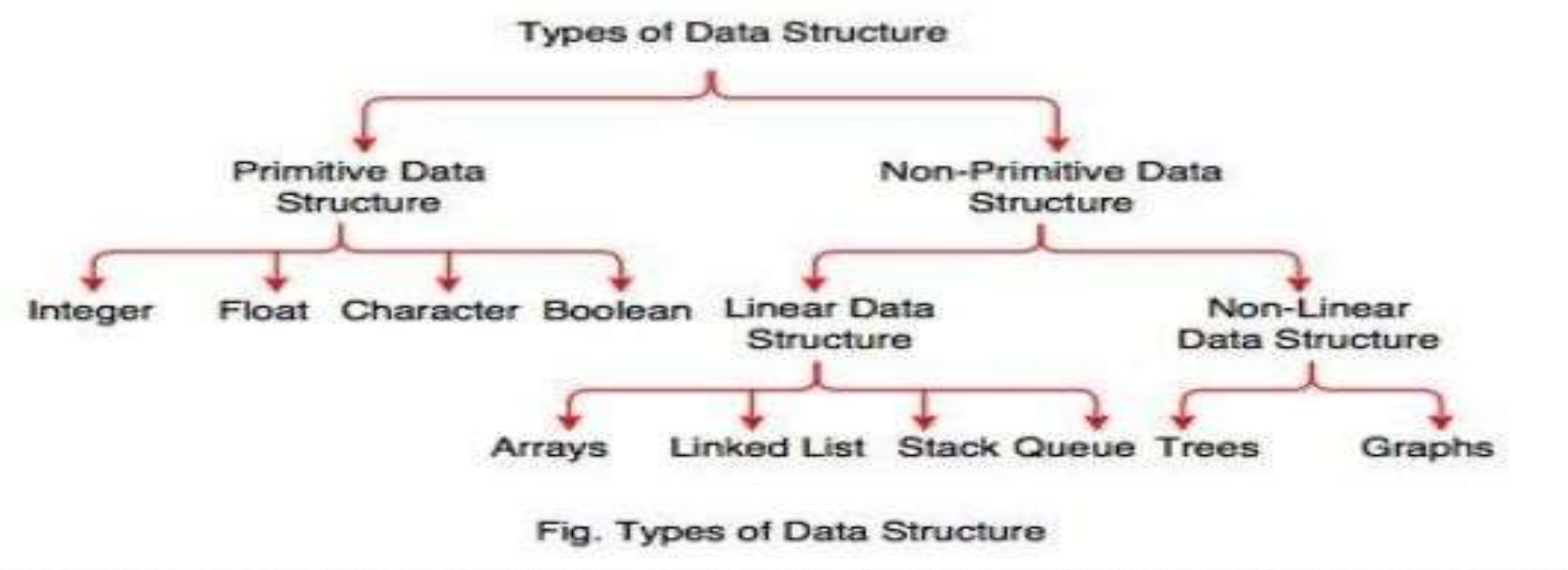
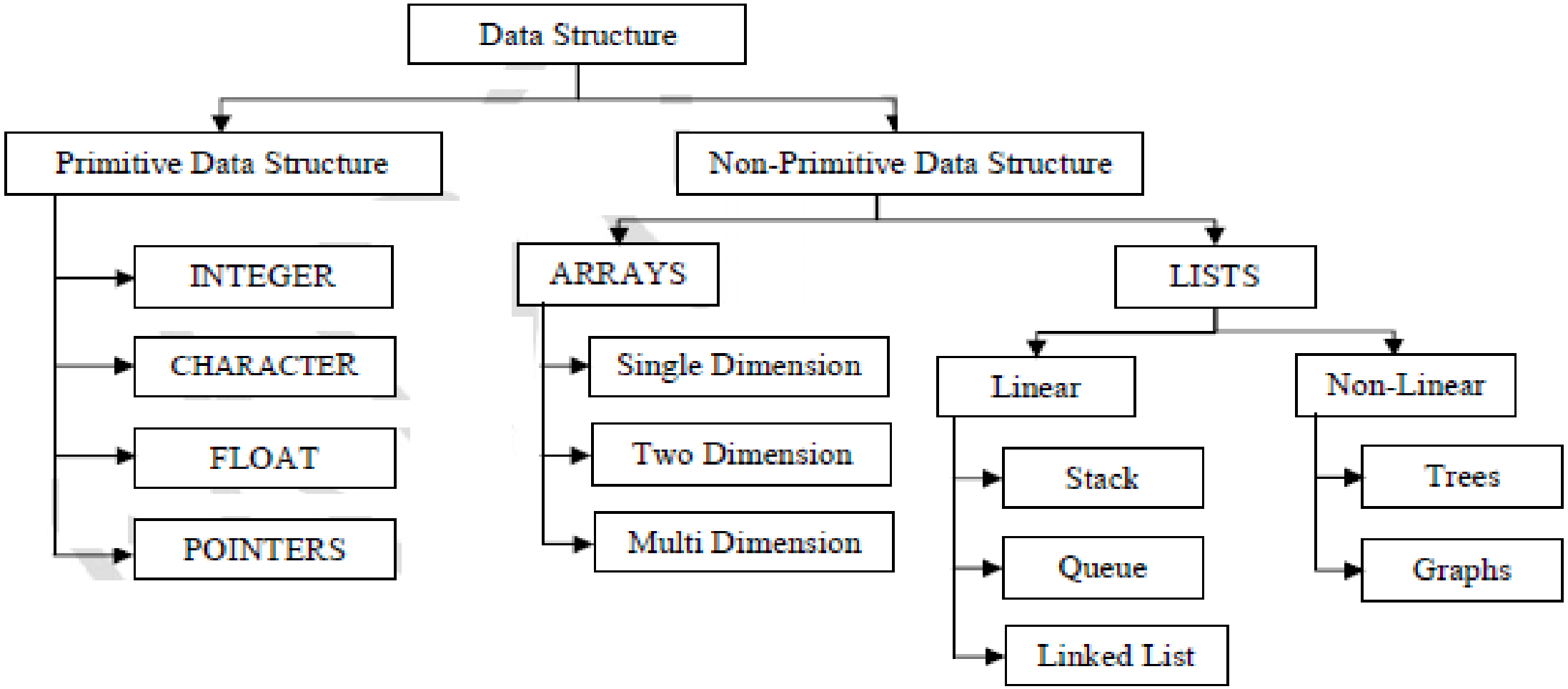


Fig. Types of Data Structure

Classification of Data Structure



Primitive Data Structure

- There are basic structures and directly operated upon by the machine instructions.
- ***Data structures that are directly operated upon the machine-level instructions are known as primitive data structures.***
- Integer, Floating-point number, Character constants, string constants, pointers etc, fall in this category.

Primitive Data Structure

- The most commonly used operation on data structure are broadly categorized into following types:
  - Create
  - Selection
  - Updating
  - Destroy or Delete

Non-Primitive Data Structure

- There are more d data structures.
- ***The Data structures that are derived from the primitive data structures are called Non-primitive data structure.***
- The non-primitive data structures emphasize on structuring of a group of homogeneous (same type) or heterogeneous (different type) data items.

DATA STRUCTURE

- **Non Primitive D.S**-
  - Used to store a group of values
  - More Complex
  - Derived from primitive data structures
  - Array, Stack, Queue, Linked list, Tree, Graph

depending upon structure and arrangement, further classified as **Linear** and **Non Linear**

Non-Primitive Data Structure

Linear Data structures:

- *Linear Data structures are kind of data structure that has homogeneous elements.*
- The data structure in which elements are in a sequence and form a liner series.
- Linear data structures are very easy to implement, since the memory of the computer is also organized in a linear fashion.
- Some commonly used linear data structures are **Stack, Queue and Linked Lists.**

Non-Linear Data structures:

- *A Non-Linear Data structures is a data structure in which data item is connected to several other data items.*
- Non-Linear data structure may exhibit either a hierarchical relationship or parent child relationship.
- The data elements are not arranged in a sequential structure.
- The different non-linear data structures are **trees and graphs.**

DATA STRUCTURE

- **Linear Data Structure**
  - Elements form a sequence or a linear list.
  - Data is arranged in a linear fashion
  - The way they are stored in memory need not be sequential
  - Array, Stack, Queue, Linked list.
- **Non Linear Data Structure**
  - Elements are not arranged sequentially.
  - Tree, Heap, Graph.

Operations On D.S

- **Data Structure Operations**: The data appearing in data structures are processed by means of operations :

Some most frequently used operations are:

Traversing: Accessing each record exactly once.

Searching: Finding the location of the record with a given key value or finding the location of all records satisfying certain conditions.

Inserting: Adding a new record to the structure.

Deleting: Removing a record from the structure.

DATA STRUCTURE

Following two operations are used in special situations:

Sorting: Arranging the records in some logical order.

Merging: Combining the records in two different sorted files into a single sorted file.

The most commonly used operation on data structure are -

- Create
- Selection
- Updating
- Searching
- Sorting
- Merging
- Delete

THANK YOU